

ContextAPI e gerenciamento de estados

Em aplicações React modernas, quando queremos repassar estados e dados de um componente filho para um componente pai, utilizamos as **props** para isso:

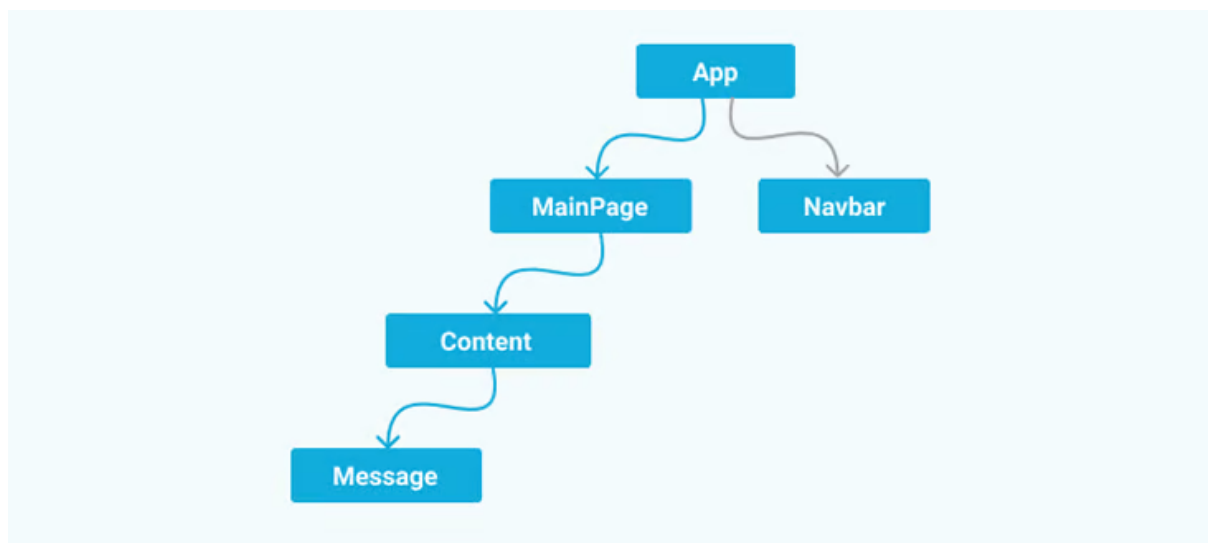
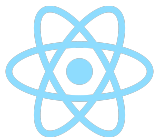
```
export function ComponentePai () {  
  return (  
    <ComponenteFilho name="Fulano" />  
  )  
}
```

```
type TFilho {  
  name: string  
}  
  
export function ComponenteFilho (props: TFilho) {  
  return (  
    <div>{props.name}</div>  
  )  
}
```

O Prop Drilling

Entretanto, imagine que nossa aplicação cresça de tamanho e se torne mais complexa, exigindo que **vários componentes recebam informações semelhantes**. Por exemplo, imagine que a tela de uma rede social exija que apresentemos o nome e a foto do usuário em vários locais distintos. Isso seria muito desafiador e provavelmente faríamos algo não recomendado em aplicações React: **Prop Drilling** (algo traduzido como perfuração de propriedades).

O Prop Drilling é a prática de criar cascatas de props em componentes. Isso se torna um problema, pois no meio do caminho teríamos componentes que não precisam receber essas propriedades e só estão servindo como "repasse" das informações. Para contornar isso, os desenvolvedores do React criaram a **ContextAPI**.



No exemplo, precisamos exibir uma informação no componente *Message*, porém ela só está acessível a partir do topo no componente *App*. Para isso, há um repasse da propriedade para o componente *MainPage*, que por sua vez repassa para o componente *Content*, que por fim encaminha a *Message*. Não há a necessidade de *MainPage* e *Content* terem essa propriedade, porém eles estão a recebendo.

ContextAPI

Introduzido na versão 16 do React, a **ContextAPI** surgiu para ajustar os problemas do *prop drilling*, como uma forma de passar os dados entre componentes sem a necessidade de repasse das propriedades. Apenas os componentes inscritos receberão os dados, sem componentes que não precisam dessas informações.

Aqui um exemplo de como um componente se inscreve para receber as informações do contexto:

```
// import useContext

function UserProfile() {
  const userDetails = useContext(UserContext);
  // rest of the component
}
```

Aprenderemos como criar um contexto e inscrevê-lo nos componentes filhos nos vídeos práticos, veja os conteúdos adicionais para estudo:

<https://react.dev/learn/passing-data-deeply-with-context>

<https://blog.logrocket.com/react-context-api-deep-dive-examples/>